LLM REASONING CM1 / 120 minutes

Reasoning at inference time

Sampling, verification and search with a fixed model

Mohamed Imed Eddine GHEBRIOUT

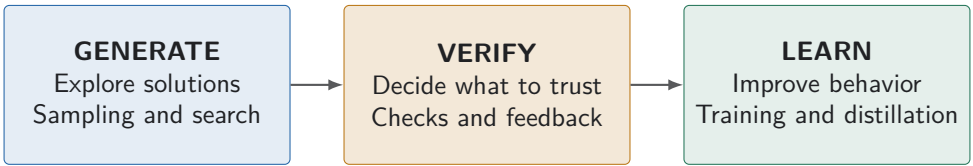
mohamed-imed-eddine.ghebriout@univ-lorraine.fr

IDMC, Université de Lorraine

2026–2027

LLM Reasoning: from answers to reliable systems

THE MODULE QUESTION How can we generate, check and improve solutions?



Understand the mechanisms
Follow a generated solution, a verification decision and a parameter update.

Build evidence for your claims
Measure correctness, failures and computation under a declared protocol.

Follow the complete system: how it proposes, checks, learns and acts.

The module at a glance

10 H LECTURES + 10 H LABS Six sessions; concepts paired with experiments.

PART 1 / FOUNDATIONS

Mohamed Imed Eddine GHEBRIOUT

Lecture + corresponding lab	CM + TD
1. Inference-time reasoning	2 h + 2 h
2. Verification and latent reasoning	2 h + 2 h
3. Learning from verification	1 h + 1 h

5 h lectures + 5 h labs

PART 2 / CONTINUATION

Estelle ZHENG

Lecture + corresponding lab	CM + TD
4. Reinforcement learning for reasoning	2 h + 2 h
5. On-policy distillation	2 h + 2 h
6. Agents and tool use	1 h + 1 h

5 h lectures + 5 h labs

From generation and verification to learning and agentic systems.

Your first three sessions: concepts into experiments

OUR ROUTE Each lab turns a lecture distinction into observable evidence.

Session	In the lecture	In the lab
1 / Generate 2 h CM + 2 h TD	Sampling, voting, search and inference cost	Extract, vote and compare policies under a token budget.
2 / Verify 2 h CM + 2 h TD	Verifier targets, feedback and latent reasoning	Build an exact checker and compare revisions with feedback.
3 / Learn 1 h CM + 1 h TD	Rewards, gradients and policy updates	Update a CPU policy and diagnose a flawed reward.

Working format: Colab | Pairs | CPU route provided

Bring: Python functions and notebooks, basic probability and gradients.

Course & lab materials:
https://github.com/Mohamed-Imed-Eddine/LLM_Reasoning-IDMC-2026_2027

Today: improve the inference procedure while keeping model parameters fixed.

The system around a language model

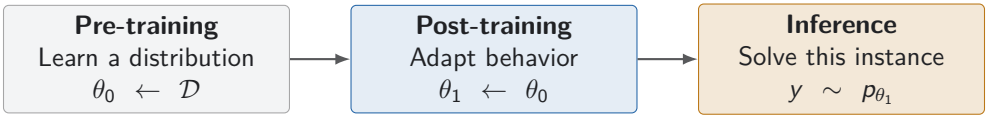


Question	What you will be able to do
Generate	Explain how decoding changes the candidate distribution.
Select	Separate oracle coverage from the accuracy of the returned answer.
Allocate	Compare sampling, revision and search under a complete budget.

A stronger candidate pool helps only if the system can use it.

Source: Stanford CS329A, Parts 1–2.

Three scales of computation



Amortized cost

Training changes parameters shared across future questions.

Per-question cost

Inference spends computation on a particular input and candidate pool.

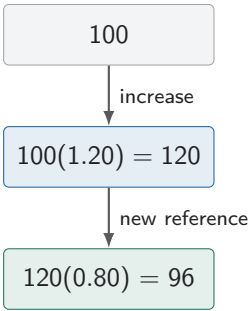
Discuss: When can extra inference compute compensate for a smaller model?

Source: Snell et al. (2024), test-time compute allocation.

A running example: the reference quantity matters

Problem
A price starts at €100, increases by 20%, then decreases by 20%. What is the final price?

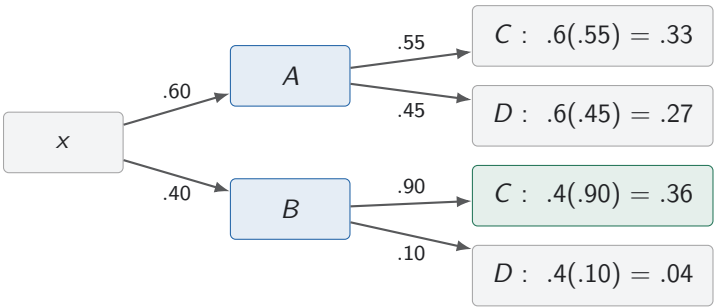
Tempting shortcut:
$+20\% - 20\% = 0$.



The trace exposes intermediate claims that a checker can inspect.

A language model assigns probability to a trajectory

CHAIN RULE Multiply along a path; condition on the prefix.



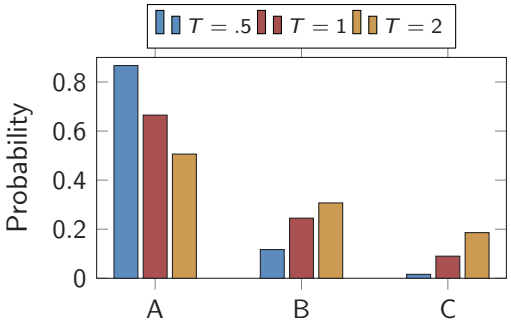
Greedy: AC

Most likely complete path: BC

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid x, y_{<t}), \qquad \log p_\theta(y \mid x) = \sum_{t=1}^{T} \log p_\theta(y_t \mid x, y_{<t})$$

Token-level greedy choices need not maximize sequence probability.

Temperature redistributes mass over tokens



$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Logits: $(2, 1, 0)$.

Lower temperature concentrates mass. Higher temperature exposes more alternatives.

Greedy decoding chooses an argmax rather than drawing a sample.

Diversity creates alternatives; it does not add knowledge.

Nucleus sampling (top-p) truncates the distribution

Token	p_i	Cumulative
A	.50	.50
B	.30	.80
C	.15	.95
D	.05	1.00

For threshold $q = .8$, retain A and B.

$$m = \min \left\{ j : \sum_{i=1}^{j} p_{(i)} \geq q \right\}$$

$$\widetilde{p}_{(i)} = \frac{p_{(i)} \mathbf{1}[i \leq m]}{\sum_{j=1}^{m} p_{(j)}}$$

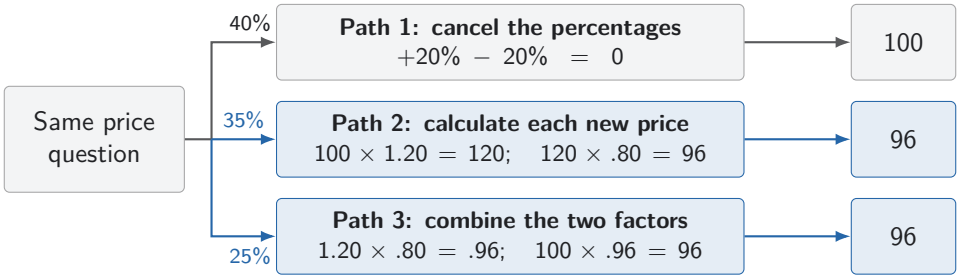
Retained probabilities: .625 and .375.

Top-k fixes the number retained; top-p adapts it to the distribution.

Discuss: Which tokens survive when $q = .9$?

Self-consistency: group paths by their final answer

1 / 2 FOLLOW THE PATHS Three possible complete solutions to the same question.



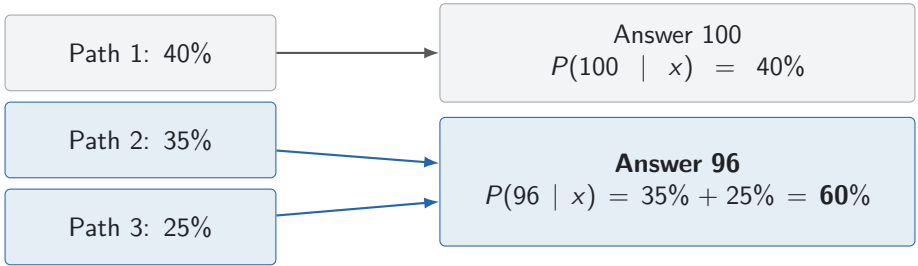
Labels give **complete-solution probabilities**; final answers are extracted.

The most likely individual solution is Path 1: 40%, ending in 100.

Source: Wang et al., Self-Consistency.

Self-consistency: group paths by their final answer

2 / 2 GROUP MATCHING ANSWERS Different solutions can support the same final answer.



Keep only the most likely solution
Return 100: its path has probability 40%.

Combine support across solutions
Return 96: its paths total 60%.

Self-consistency: sample solutions, extract their answers, then vote.

Voting approximates this grouping; repeated agreement can still support a wrong answer.

Source: Wang et al., Self-Consistency.

Prompting changes context; post-training changes parameters

Few-shot CoT prompting

Example question
Worked intermediate steps
Example answer
New question

A model conditions on demonstrations.

$$\theta' = \theta - \eta \nabla_\theta \mathcal{L}$$

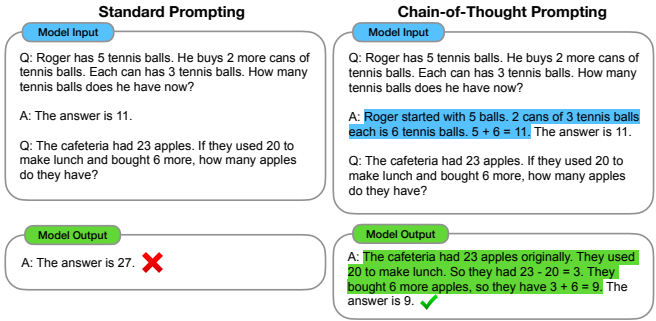
Post-training

SFT or RL changes the distribution from which trajectories are generated.

A reasoning-trained model and an ordinary model given a CoT prompt are different experimental interventions.

Source: Wei et al. (2022), Chain-of-Thought Prompting.

What changed in the original CoT experiment?

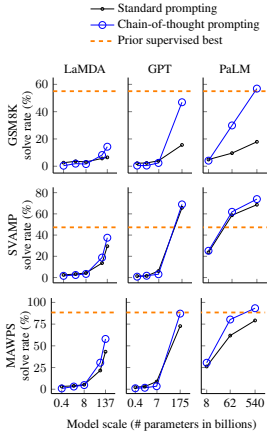


The demonstration includes intermediate calculations. The test question stays the same.

Discuss: Which intervention would isolate reasoning content from extra output length?

Source: Wei et al., Figure 1. Original prompt and outputs.

CoT gains depend on model scale and task



Compare within a panel
Same model family and benchmark.

Then compare panels
The benefit varies across tasks and scales.

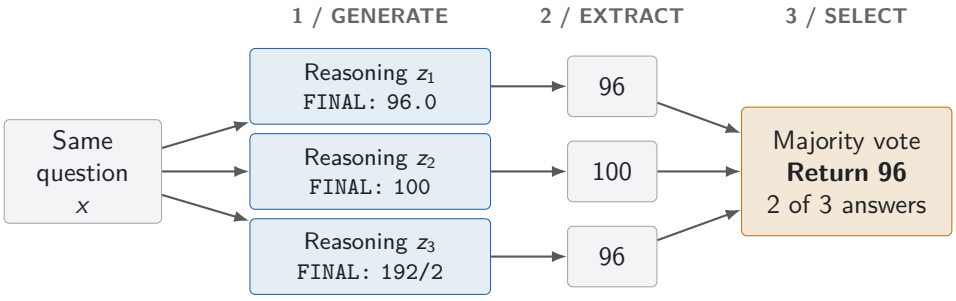
These are historical model families.

Validate the prompting strategy on your model and task.

Source: Wei et al., Figure 4.

From one trajectory to a candidate pool

ONE QUESTION, THREE ATTEMPTS Each candidate is a complete generated response.



$$\mathcal{Y}_k(x) = (y^{(1)}, \ldots, y^{(k)}), \qquad y^{(i)} \sim q_\theta(\cdot \mid x)$$

k counts generated responses. Same model and prompt; q_θ also includes the decoding settings. Keep duplicates and failed outputs in the experiment.

Generating a pool and choosing its returned answer are separate steps.

Coverage and selection answer different questions

WORKED EXAMPLE Three questions; four candidates each.

QUESTION	SAMPLED ANSWERS				REF.	COVERED?	VOTE RETURNS
Q1	19	19	17	19	19	Yes	19
Q2	100	100	96	100	96	Yes	100
Q3	8	8	6	8	7	No	8

Green candidates match the reference; reference labels are used for evaluation only.

Coverage: 2/3
Q2 contains a success: selection misses it.

Vote accuracy: 1/3
Q3 contains no success: selection alone cannot fix it.

Availability is necessary; returning the right candidate is a separate task.

What does pass@k measure?

FROM A COUNT TO AN ESTIMATOR Keep the subset experiment separate from selection.

One observed pool

$n = 5$ candidates; $c = 2$ correct. Choose $k = 2$ uniformly without replacement.



All pairs: $\binom{5}{2} = 10$. All-wrong pairs: $\binom{3}{2} = 3$.

$$\widehat{C}_2 = 1 - \frac{3}{10} = \boxed{.70}$$

C = correct; W = wrong. Labels are evaluation-only.

Generalize the count

$$\widehat{C}_k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

Chance that a random k-subset contains a success; average across questions.

Combinations: $\binom{n}{k} = C_n^k = \frac{n!}{k!(n-k)!}$

Choose k from n: no order, no replacement ($0 \le k \le n$).

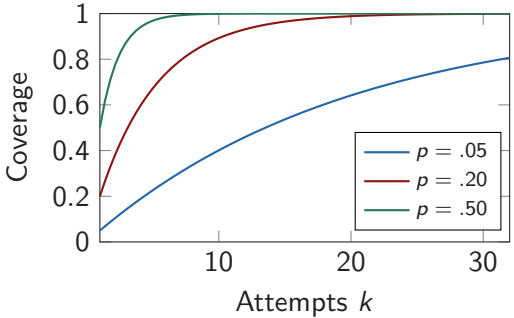
Independent-draw plug-in: $\hat{p} = 2/5$ gives $1 - (1 - \hat{p})^2 = .64$.

Assumptions: $0 \le c \le n$, $1 \le k \le n$; $\binom{n-c}{k} = 0$ if $k > n - c$.

Neither number is the accuracy of a deployable selector.

Source: Brown et al. (2024), coverage.

A simple coverage law, and its assumptions



$$C_k = 1 - (1 - p)^k$$

Independent attempts, fixed per-question success probability p.

$$C_{k+1} - C_k = p(1 - p)^k$$

Marginal gain decreases as the successful region is covered.

Use the per-question success rate to predict the gain from another attempt.

Averaging difficulty changes the coverage curve

Two equally frequent question types
Easy: $p_e = .8$
Hard: $p_h = .02$

Mean success probability: $\bar{p} = .41$.

Correct mean coverage at $k = 4$

$$\overline{C}_4 = \tfrac{1}{2}(1 - .2^4) + \tfrac{1}{2}(1 - .98^4) \approx .538.$$

Using the mean probability: overestimate

$$1 - (1 - .41)^4 \approx .879$$

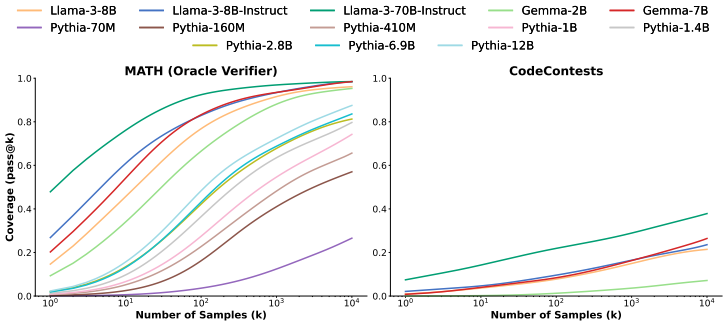
Treats all questions as equally difficult.

Correct: average per-question coverage

$$\overline{C}_k = \frac{1}{N} \sum_{j=1}^{N} \left[1 - (1 - p_j)^k \right]$$

Calculate coverage for each question, then average.

Repeated sampling: evidence across model families



Read the axes: log sample count; fraction of problems with a correct candidate.

Scope: MATH and CodeContests, models and prompts studied in the paper.

Source: Brown et al. (2024), Figure 3.

Self-consistency approximates an answer distribution

SAMPLE → NORMALIZE → COUNT → DECIDE

TD1 tie rule.

$$\widehat{p}(a \mid x) = \frac{1}{k} \sum_{i=1}^{k} \mathbf{1}[\text{extract}(y^{(i)}) = a], \qquad \hat{a} \in \arg\max_{a \neq \perp} \widehat{p}(a \mid x).$$

Sample	FINAL value	Canonical answer
1	`42.0`	42
2	`84/2`	42
3	missing	$\perp$ (invalid)
4	17	17
5	17	17

Two votes each: 42 **and** 17

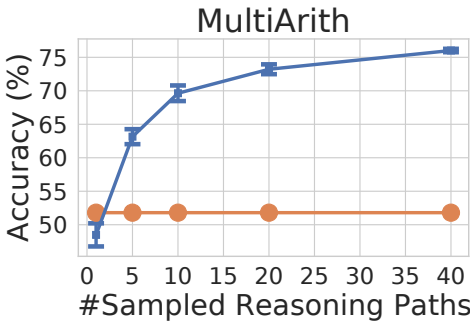
Ignore invalid outputs when voting. Resolve the tie by first appearance: return 42.

If every output is invalid, abstain. Keep that question in the accuracy denominator.

A vote count measures agreement, not the probability of being correct.

Source: Wang et al., Self-Consistency.

Self-consistency: historical empirical results



PROTOCOL

LaMDA-137B on MultiArith. Vary the number of sampled paths.

READ

Blue: self-consistency.

Orange: greedy.

Mean and standard deviation over 10 runs.

LIMIT

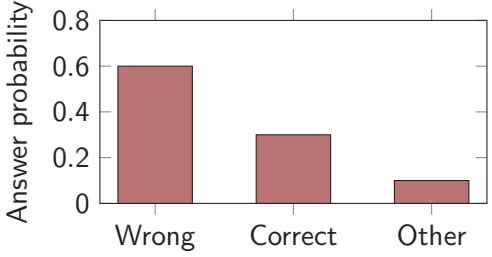
Path count is not total compute.

Selected-answer accuracy is not oracle coverage.

Discuss: How much does the last doubling of samples improve the selected answer?

Source: Wang et al., Figure 2, MultiArith panel; full figure retained in source assets.

Majority vote can converge to the wrong answer



As k increases, empirical frequencies converge to these probabilities.

The most frequent answer remains wrong.

Yet oracle coverage tends to 1 because the correct answer has positive mass.

More samples can strengthen the wrong consensus.

Best-of-N uses a learned or executable score

$$\hat{y} = \arg \max_{y \in \mathcal{Y}_N(x)} r_\phi(x, y).$$

Candidate	Answer	Score
y_1	100	.34
y_2	96	.91
y_3	100	.63

Generator: supplies the pool.

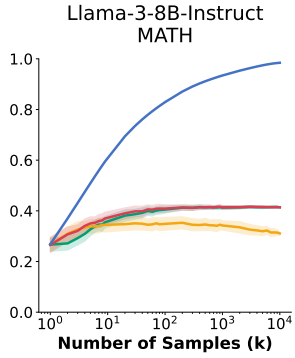
Verifier: determines the ranking.

Evaluator: checks the returned answer against held-out ground truth.

The score's meaning and reliability determine the selection policy.

Source: Cobbe et al. (2021), Training Verifiers.

The generation–verification gap in real experiments



Llama-3-8B-Instruct / MATH

Blue: oracle coverage.
Green: majority vote.
Orange: reward-model best-of-N.
Red: reward-model weighted vote.

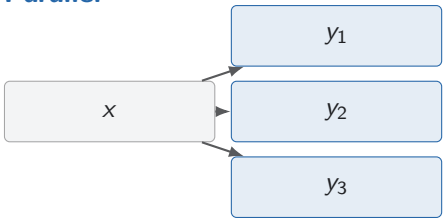
Read the log sample axis and success rate. More samples expose correct candidates; these selectors leave much of the gain unused.

The gap is available potential, not a promised verifier gain.

Source: Brown et al. (2024), Figure 7, Llama-3-8B-Instruct / MATH panel; shared legend restated.

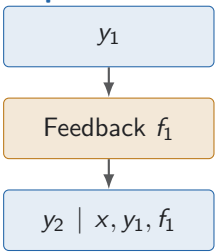
Parallel sampling and sequential revision

Parallel



Explores alternative initial trajectories.

Sequential



Later attempts depend on earlier ones.

Source: Snell et al. (2024).

Revision must be evaluated, not assumed to help

Model	CC. Test		HumanEval+		MBPP+	
	ST	MT	ST	MT	ST	MT
Llama 3.1 8B Instruct	11.8	10.5	65.3	63.9	58.3	60.5
+ RLEF	9.7	16.0	67.5	69.5	57.0	63.1
Llama 3.1 70B Instruct	26.2	27.4	73.2	75.0	66.9	70.2
+ RLEF	30.3	40.1	78.6	80.4	67.6	72.2
gpt-4o-2024-05-13	25.3	24.3	82.8	80.7	68.8	71.7

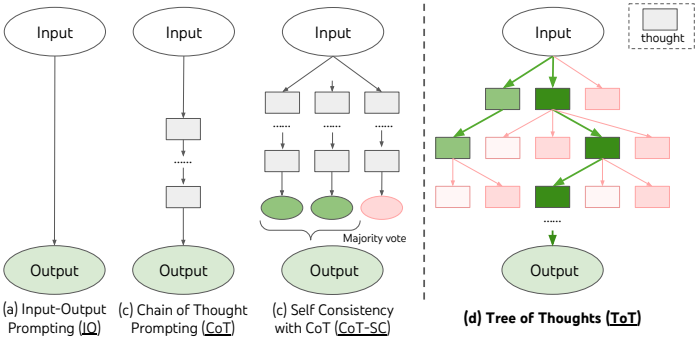
Read the first two rows. CodeContests, same three-response budget:
8B Instruct: 11.8 → 10.5 (decrease). **After RLEF:** 9.7 → 16.0 (increase).

Using feedback is a learned behavior; prompting alone need not improve it.

Discuss: Which comparison changes inference, and which also changes the model?

Source: RLEF, Table 2; 1@3 solve rates (%), ST = single-turn, MT = multi-turn.

Search exposes decisions before the final answer



State: partial reasoning.

Action: next thought.

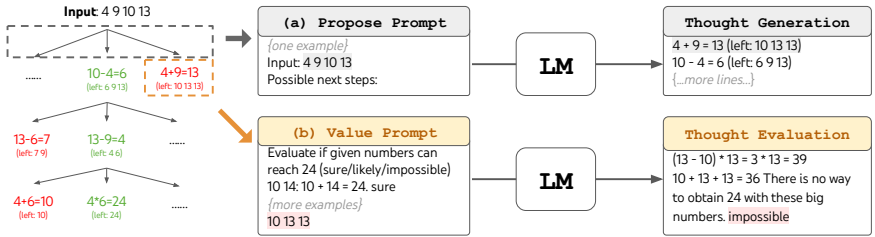
Value: promise of a prefix.

Control: expand, prune, backtrack.

The search policy decides where future computation goes.

Source: Yao et al. (2023), Tree of Thoughts, Figure 1.

Tree of Thoughts separates proposal from evaluation



Proposal prompt

Suggest arithmetic transitions using the remaining numbers.

Value prompt

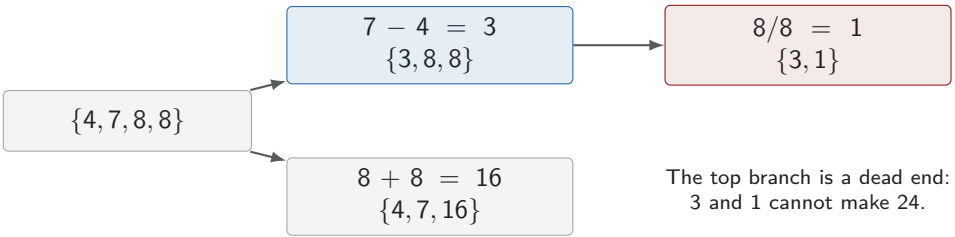
Assess whether a partial state can still reach 24.

The value model guides allocation. The final checker must enforce arithmetic and number-use constraints.

Source: Yao et al., Figure 2.

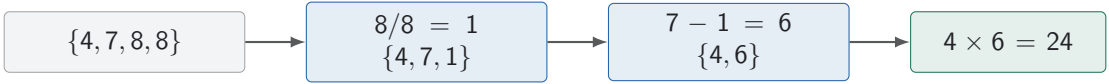
A concrete search state: the Game of 24

Use $4, 7, 8, 8$ exactly once, with arithmetic operations, to obtain 24.



A locally plausible operation can lead to an unsolvable state.

A successful trajectory keeps the useful intermediate state



$$4\left(7 - \frac{8}{8}\right) = 24.$$

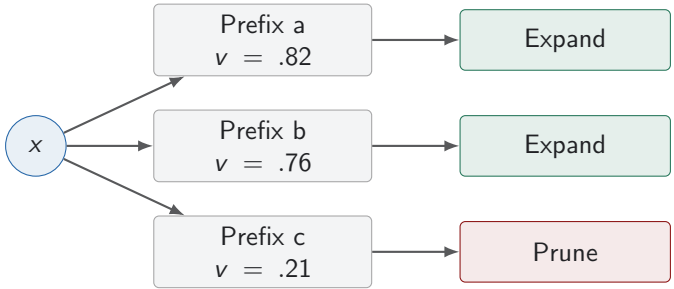
Transition check

Consume exactly the operands used and insert their result.

Terminal check

Require one remaining value, equal to the target, with every input used once.

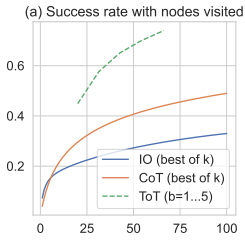
Beam search trades breadth for early commitment

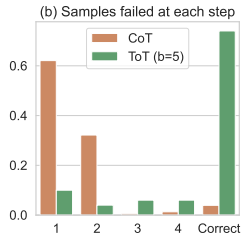


With beam width 2, a scoring error can permanently remove the correct path.

Tree of Thoughts: the result and the comparison

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	**74%**
IO + Refine (k=10)	27%
IO (best of 100)	33%
CoT (best of 100)	49%





Observed

100 Game of 24 instances.

ToT with $b = 5$: 74% success.

Search includes proposal and evaluation calls.

b = search breadth (beam width)

Retain up to **5 partial states per level**.

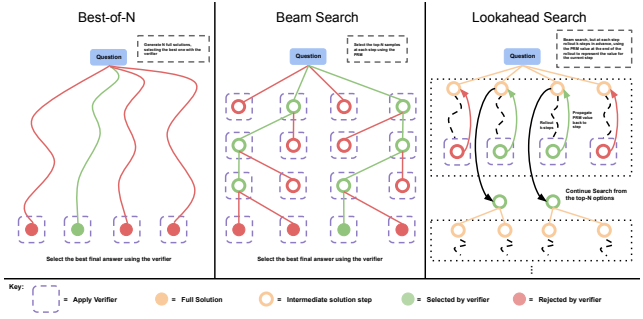
Propose $\rightarrow$ score $\rightarrow$ keep best b

$\rightarrow$ expand the retained states.

The best-of-100 CoT row uses an oracle success check. It differs from majority-vote CoT-SC.

Source: Yao et al., Table 2 and Figure 3.

Three ways to spend verifier computation



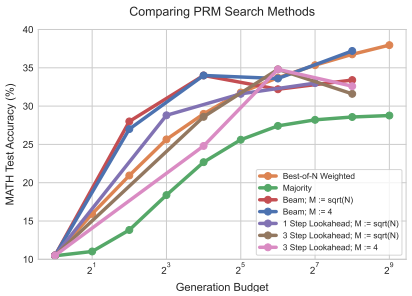
Best-of-N
Generate complete solutions, then score them.

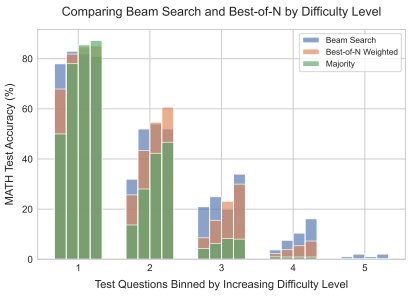
Beam search
Score the current partial solutions; retain the best.

Lookahead: look k steps ahead
Generate up to k extra steps; use the later score to rank the current partial solution.

Source: Snell et al. (2024), Figure 2.

Search quality changes with difficulty and budget





Aggregate curve: beam search gains early, then its advantage shrinks.

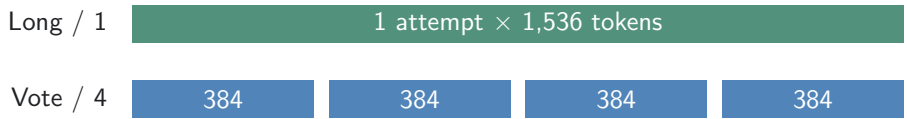
Difficulty bins: easy and medium questions favor different strategies.

A larger search budget can also exploit errors in the process scorer.

Source: Snell et al., Figure 3.

Compute accounting includes the whole system

TD1 BUDGET Same maximum output allowance; different computation.



The allowance is a ceiling

Actual outputs may stop early. Count tokens from every attempt, including thinking and unsuccessful outputs.

The system also does other work

Four attempts process the prompt four times. Verification, cache reuse and batching affect time and compute.

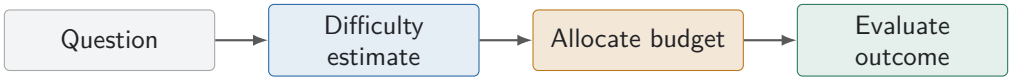
$$C_{\text{total}} = C_{\text{generation}} + C_{\text{verification}} + C_{\text{control}} + C_{\text{tools}}$$

Use one cost unit for this sum; count each operation once. Tokens, seconds and FLOPs are separate reports.

Compare accuracy under the cap; report the actual cost separately.

Adaptive allocation is a decision problem

$$s^*(x; B) = \arg \max_{s:\, C(s,x) \le B} \Pr(\hat{a}_s(x) = a^* \mid x)\,.$$

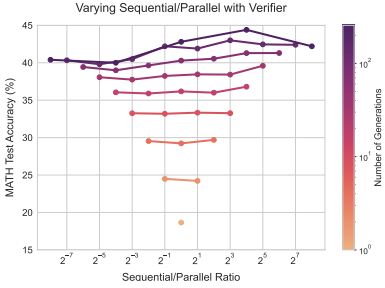


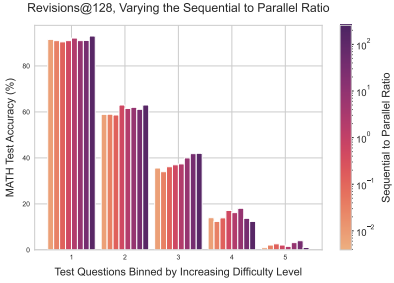
Possible actions
Another independent sample; deeper revision; wider search; stronger verification.

Unknown at deployment
True difficulty and the best strategy. Estimate them on development data.

Source: Snell et al. (2024), compute-optimal inference.

A fixed budget can mix revisions and fresh attempts





Each curve holds total generation budget fixed. The horizontal axis changes its allocation.

At budget 128, the best ratio varies by difficulty. A single universal ratio loses information.

Source: Snell et al., Figure 7.

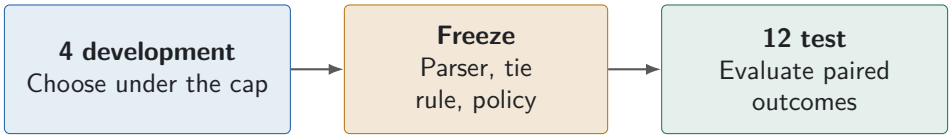
Choosing an inference method from the task structure

Available structure	Candidate method	What to validate
Canonical answer, many paths	Self-consistency	Extraction and wrong-answer modes
Executable final check	Sampling + checking	Test coverage and execution cost
Useful prefix evaluation	Beam / tree search	False pruning and score exploitation
Informative error feedback	Sequential revision	Repair rate and regressions
Different question difficulties	Adaptive allocation	Routing overhead and held-out gains

Discuss: Where would code generation and a free-form essay fall in this table?

An experiment should isolate generation from selection

TD1 PROTOCOL Decide on development questions; report once on held-out questions.



To compare selectors
Use the same frozen candidate pools. Change only the rule that returns an answer.

To compare generation strategies
Keep questions and the declared budget fixed. Report all protocol differences and costs.

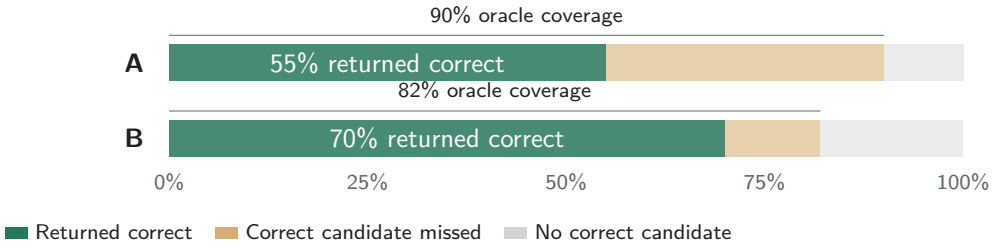
Report: returned accuracy, oracle coverage, actual costs and failed outputs. Resample paired questions, not individual candidates.

A change made after seeing test outcomes needs fresh evaluation data.

Checkpoint: which system would you deploy?

PAIR CHECK 90 seconds to decide, then compare explanations.

Same 200 held-out questions; **different pools**. Output tokens / question: 800 for both.



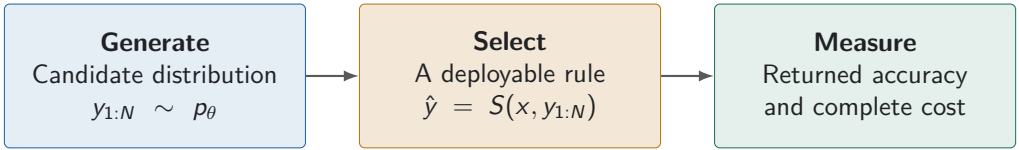
1 / Decide
Which system returns more correct answers? How many more?

2 / Diagnose
Have we isolated a better selector? What must be held fixed?

3 / Audit
Does matching output tokens establish equal compute?

From inference scaling to verification

CM1 SYNTHESIS A fixed model; three system-level choices.



Keep separate: probability / correctness coverage / selection tokens / compute

TD1 / 2 h Implement extraction, voting, coverage and a budget choice.

Evidence to keep: A cost–accuracy comparison and two diagnosed traces; select without gold labels.

Next in CM2 What makes a verification signal trustworthy? What can be checked when intermediate states are not text?

The next bottleneck is the quality of the verification signal.

Reading map

Stanford CS329A: Part 1, 19:32–24:12; Part 2, 15:34–19:16 and 30:02–41:07.

Wei et al. (2022), Chain-of-Thought Prompting.
Wang et al., Self-Consistency.
Brown et al. (2024), Large Language Monkeys.
Yao et al. (2023), Tree of Thoughts.
Snell et al. (2024), Scaling LLM Test-Time Compute Optimally.

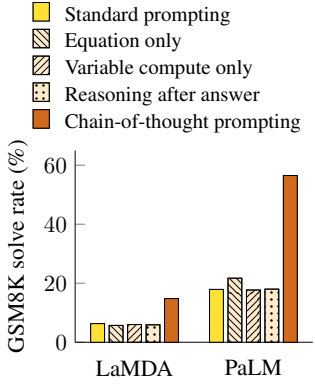
Core reading: Brown et al., Sections 2 and 4. Extension: compute allocation in Snell et al.

Further study

Research details and discussion

Optional slides beyond the scheduled lecture.

CoT ablations test competing explanations



Equation only
Intermediate mathematical expressions.

Variable compute only
Extra dots rather than useful steps.

Reasoning after answer
Explanation follows the commitment.

The original controls distinguish useful intermediate content from mere output length.

Source: Wei et al., Figure 5.

Tree search as an explicit algorithm

Algorithm 1 ToT-BFS($x, p_\theta, G, k, V, T, b$)

Require: Input x, LM p_θ, thought generator $G()$ & size limit k, states evaluator $V()$, step limit T, breadth limit b.

$S_0 \leftarrow \{x\}$

for $t = 1, \cdots, T$ **do**

$S'_t \leftarrow \{[s, z] \mid s \in S_{t-1}, z_t \in \mathrm{G}(p_\theta, s, k)\}$

$V_t \leftarrow V(p_\theta, S'_t)$

$S_t \leftarrow \arg\max_{S \subset S'_t, |S|=b} \sum_{s \in S} V_t(s)$

end for

return $G(p_\theta, \arg\max_{s \in S_T} V_T(s), 1)$

G: proposes thoughts.

V: evaluates states.

b: beam width.

T: depth limit.

The controller keeps a set of prefixes at each depth.

Search requires a state representation and a termination rule.

Source: Tree of Thoughts, Algorithm 1.

Best-of-N and weighted voting can return different answers

Candidate	Extracted answer	Verifier score
y_1	96	.60
y_2	96	.60
y_3	100	.90

Best-of-N

$\arg\max_i r_i$ selects y_3, answer 100.

Answer-level weighted vote

$\arg\max_a \sum_i \mathbf{1}[a_i = a] r_i$ selects 96.

Sum aggregation rewards support across paths. Scores must have a meaningful nonnegative scale for this interpretation.

Source: Math-Shepherd, Section 3.4, Equation 5.

A search budget counts both proposals and evaluations

Suppose depth $d = 4$, beam width $b = 3$, and $k = 5$ children per retained state.

$$N_{\text{proposed}} \leq k + (d-1)bk = 5 + 3 \cdot 3 \cdot 5 = 50.$$

With one scorer call per child
Up to 50 evaluations, before checking final outputs.

With m rollout probes per child
Up to $50m$ additional continuation generations.

Prefix reuse and batching affect actual cost. A node count is a useful accounting unit, not a complete FLOP estimate.

Stopping depends on marginal value and cost

With independent attempts of success probability p:

$$\Delta C_k = C_{k+1} - C_k = p(1-p)^k.$$

If an extra attempt costs c and a correct answer has utility u, an oracle calculation continues while

$$u\,p(1-p)^k > c.$$

At $p = .2$, the gain is .0819 for the fifth attempt and .0215 for the eleventh. A real controller must also account for selection errors and uncertainty about p.

Difficulty estimates are part of the evaluation protocol

Oracle difficulty

Check model samples against reference answers.

Useful for analyzing the best achievable allocation.

Predicted difficulty

Estimate from model or verifier signals without the answer.

Requires validation and extra inference work.

Snell et al. estimate model-specific difficulty using 2,048 samples per question and use cross-validation for selecting strategies.

Discuss: Would that preprocessing cost be acceptable for one new user query?

Source: Snell et al., Section 4.

A selector comparison needs paired uncertainty

For each held-out question j, compute

$$d_j = c(x_j, \hat{y}_j^A) - c(x_j, \hat{y}_j^B), \qquad \widehat{\Delta} = \frac{1}{N} \sum_j d_j.$$

Paired bootstrap
Resample questions with their complete candidate pools and both selector outcomes.

Sampling variability
Repeat candidate generation when the claim includes robustness to generation randomness.

Keep tuning questions separate. Resampling individual candidates as though they were independent questions gives the wrong unit of analysis.

Answer normalization is part of the voting algorithm

The task asks for a probability. Five independently generated responses end with:

Raw answer	Count	Canonical value
0.5	1	1/2
50%	1	1/2
1/2	1	1/2
0.4	2	2/5

Vote over strings
The answer 0.4 wins with two votes.

Vote over normalized values
The answer 1/2 wins with three votes.

The equivalence relation determines which votes are combined.

Source: Self-Consistency, answer aggregation.

Plurality voting does not require 50 percent support

An answer distribution

Answer	Probability
Correct	.40
Wrong A	.35
Wrong B	.25

Under independent draws

The empirical proportions approach their probabilities.

The correct answer is the unique mode, despite having less than half the mass.

In the binary case, three-vote majority with single-draw accuracy p has accuracy

$$\Pr(\text{majority correct}) = 3p^2(1-p) + p^3.$$

At $p = .6$, this is .648; at $p = .4$, it is .352.

Voting amplifies the mode. Whether that helps depends on the answer distribution.

Repeated text and dependent samples are different issues

Procedure	What is repeated?	Consequence
Independent decoding	Fresh draws from the same distribution	Duplicate answers can occur naturally.
Copy one sampled answer	The same random outcome	Copies provide no new chance of success.
Branch from one prefix	A shared sampled partial solution	Outcomes can depend through that prefix.

Discuss: If four responses are identical, can you infer which procedure produced them?

Diversity of strings is observable; independence is a property of the sampling procedure.

A shared early choice can limit coverage

An initial strategy is viable with probability .5. Given a viable strategy, each continuation succeeds independently with probability .6. Otherwise none succeeds.

Four fresh strategies
Each complete attempt succeeds with probability .3.

$C_4 = 1 - (1 - .3)^4 = .7599.$

One shared strategy, four branches
Condition on the shared early choice.

$C_4 = .5[1 - (1 - .6)^4] = .4872.$

Both have single-continuation success probability .3. Their four-attempt coverage differs.

A local branch cannot recover an alternative excluded before the branch point.

Discuss: What is the coverage limit as the number of branches grows?

Revision helps only when repairs outweigh regressions

Let p be initial correctness, a the probability of repairing an incorrect answer, and b the probability of breaking a correct answer.

$$p' = p(1-b) + (1-p)a, \qquad p' - p = (1-p)a - pb.$$

Initial p	Repair rate a	Regression rate b	Revised p'
.60	.25	.10	.64
.90	.25	.10	.835

The same revision behavior can help a weaker starting pool and hurt a stronger one.

Discuss: What information would justify revising only some answers?

An executable check enables a concrete stopping rule

Assume iid attempts succeed with probability $p = .2$. A perfect, cheap checker stops at the first success, with at most $K = 5$ attempts.

Success by the cap

$\Pr(\text{success}) = 1 - (1-p)^K$

$= 1 - .8^5 = .67232.$

Expected attempts

$\mathbb{E}[N] = \sum_{j=0}^{K-1} (1-p)^j$

$= \dfrac{1-(1-p)^K}{p} = 3.3616.$

Generating all five candidates has the same coverage but always pays for five attempts. Parallel generation can have a different latency profile.

Early stopping trades scheduling flexibility for lower expected generation work.

Latency and total work answer different budget questions

Each candidate takes 2 seconds on one worker; each check takes 0.1 seconds on that worker. Four candidates are required.

Schedule	Ideal wall time	Total worker time
One worker, sequential	$4(2 + .1) = 8.4$ s	8.4 worker-s
Four workers, parallel	$2 + .1 = 2.1$ s	8.4 worker-s

Latency
How long does one user wait?

Work / resource occupancy
How much capacity does the request consume?

Real batching, memory limits, queueing and unequal output lengths change both measurements.

Returned accuracy combines coverage and selection

Suppose a selector returns a correct candidate with probability q conditional on the pool containing one. It always selects from that pool.

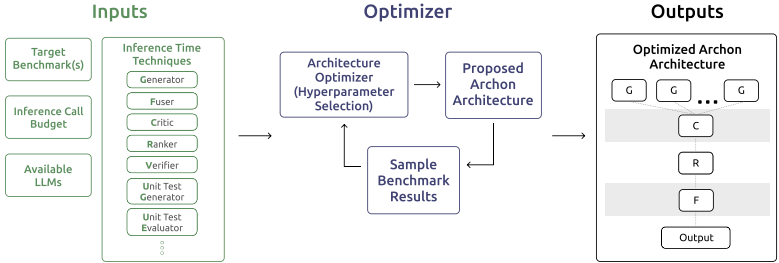
$$\Pr(\text{returned correct}) = C_k q.$$

System	Coverage C_k	Conditional selection q	Accuracy
More generation	.90	.60	.54
Stronger selection	.80	.85	.68

Generation and selection are complementary interventions, not interchangeable metrics.

Discuss: Which additional measurement is needed before choosing under a fixed budget?

Archon: the inference pipeline is a search space



PROTOCOL
Fix models, task and an inference-call budget.

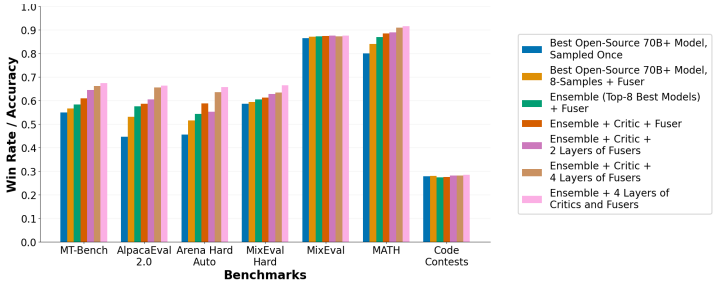
READ
Search over generators, critics, rankers and fusers.

LIMIT
Architecture tuning consumes data and compute too.

Discuss: Where should the development set end and the test set begin?

Source: Saad-Falcon et al., Archon; Figure 2: inference architecture search; PDF p. 2.

Read the variation across benchmarks



PROTOCOL

Compare inference compositions across seven benchmarks.

READ

Large changes on some tasks; little change on CodeContests.

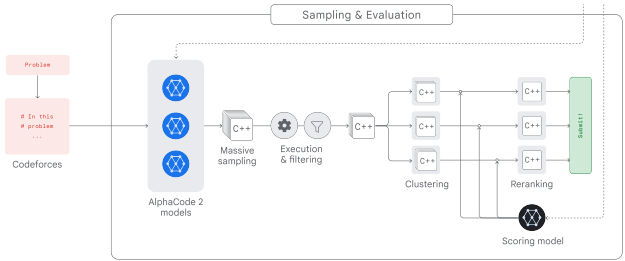
LIMIT

The vertical axis mixes task-specific metrics.

Discuss: Why is a single average gain insufficient to choose a pipeline?

Source: Saad-Falcon et al., Archon; Figure 4: seven-benchmark comparison; PDF p. 6.

AlphaCode 2: a million candidates, a few submissions



PROTOCOL
Generate many C++ solutions to a contest problem.

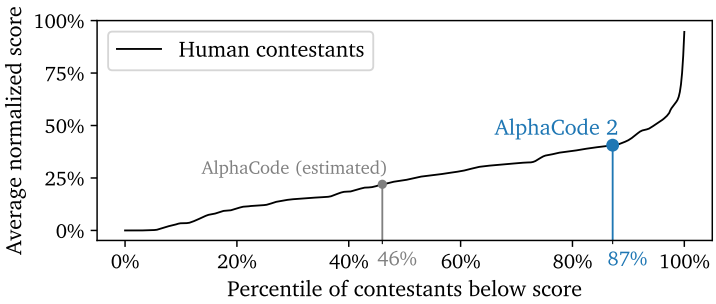
READ
Execute, filter, cluster, then rerank within clusters.

LIMIT
Passing the available tests is an incomplete check.

Discuss: Why cluster solutions after filtering them?

Source: AlphaCode Team, AlphaCode 2; Figure 1: inference pipeline (lower panel); PDF p. 2.

A percentile is not an accuracy



PROTOCOL
Twelve Codeforces contests; estimated contest rankings.

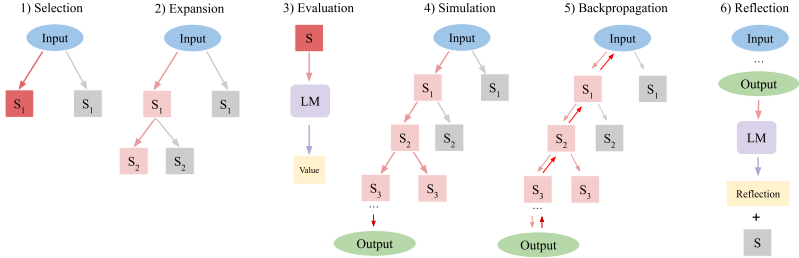
READ
Horizontal: competitors below the score. Vertical: normalized score.

LIMIT
Contest scoring and time penalties affect the estimate.

Discuss: Does the 87th percentile mean that 87% of problems were solved?

Source: AlphaCode Team, AlphaCode 2; Figure 2: normalized score and percentile rank; PDF p. 4.

LATS: turn a trajectory into a search tree



PROTOCOL
A state contains the interaction history and observations.

READ
Expand, evaluate, simulate, back up values, then reflect.

LIMIT
Real actions may be costly or irreversible.

Discuss: Does the backpropagation arrow update the language model weights?

Source: Zhou et al., LATS; Figure 2: six tree-search operations; PDF p. 5.

LATS: read the oracle hidden in the protocol

Prompt Method	HotpotQA (EM) ↑
ReAct (Yao et al., 2023b)	0.32
ReAct (best of k)	0.38
Reflexion (Shinn et al., 2023)	0.51
ToT (ReAct)	0.39
RAP (ReAct)	0.54
LATS (ReAct)	0.63
LATS ($n = 3$)	0.58
LATS ($n = 10$)	0.65
LATS (CoT + ReAct)	**0.71**

PROTOCOL
GPT-3.5; 100 HotpotQA questions; 50 sampled trajectories.

READ
The reported best row reaches 0.71 exact match.

LIMIT
The environment reveals answer correctness after an answer.

Discuss: Can this table justify the same gain without oracle feedback?

Source: Zhou et al., LATS; Table 3: acting-based HotpotQA results; PDF p. 6.